Object Oriented Programming
Lab Assignment 9

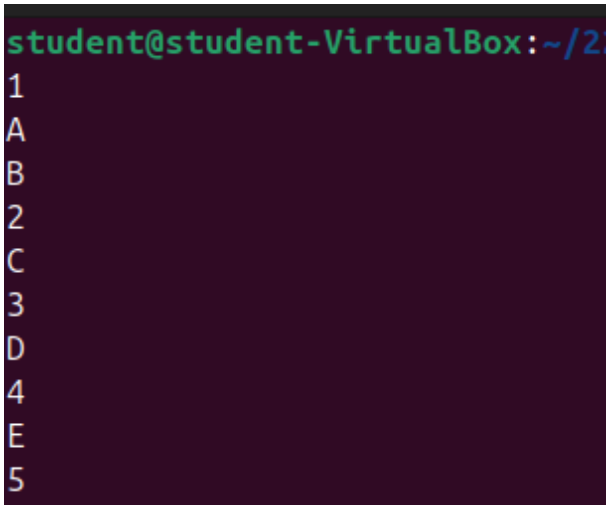
22CS02009
Kumar Snehal

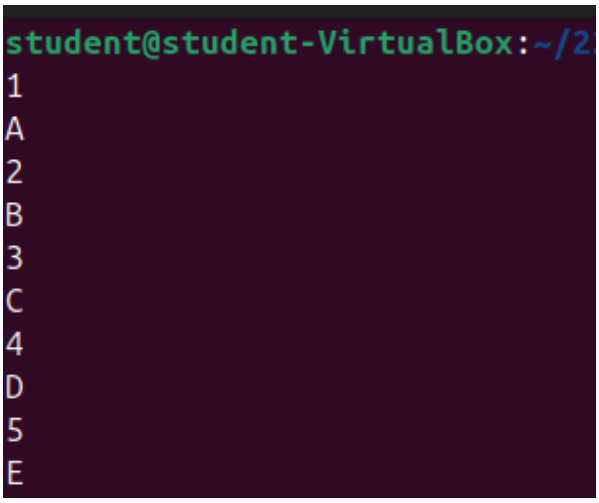
To Run:

1. Please place all the program files in a folder called “Lab_9”, this is because they were created under package name Lab_9, not doing so will give error.
2. To run program, go outside the Lab_9 directory and (# means number)
   a. `cd Lab_9`
   b. `javac *.java`
   c. `cd ..`
   d. `java Lab_9.Q#`

Problem 1:

Two classes were made: MyThread and MyRunnable, both print the parameters passed to them. ThreadA was made using MyRunnable and ThreadB was made using MyThread. A sleep of 500 ms was used.





Problem 2:

A BankAccount class was made, which had balance as a data member, and withdraw as a function.

A class WithdrawThread was made which calls withdraw of ‘x’ amount ‘t’ times. 2 Threads were made with this class.

```
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q2
Thread 4 attempting to withdraw 25 for 10 times.
Thread 2 attempting to withdraw 25 for 10 times.
Thread 3 attempting to withdraw 25 for 10 times.
Thread 1 attempting to withdraw 25 for 10 times.
Final balance:
Current balance: 0
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q2
Thread 2 attempting to withdraw 25 for 10 times.
Thread 1 attempting to withdraw 25 for 10 times.
Thread 4 attempting to withdraw 25 for 10 times.
Thread 3 attempting to withdraw 25 for 10 times.
Final balance:
Current balance: 25
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$
```

We can see that in unsynchronised code, there is a possibility of race condition, where the final balance is coming out to be incorrect as it should be 0.

```java
    public void withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
        } else {
            System.out.println("Insufficient funds. Current balance: " + balance);
```

We will solve this problem by adding the synchronised keyword to withdraw function.

```java
    public synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
        } else {
            System.out.println("Insufficient funds. Current balance: " + balance);
```

```
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q2
Thread 1 attempting to withdraw 25 for 10 times.
Thread 4 attempting to withdraw 25 for 10 times.
Thread 3 attempting to withdraw 25 for 10 times.
Thread 2 attempting to withdraw 25 for 10 times.
Final balance:
Current balance: 0
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q2
Thread 1 attempting to withdraw 25 for 10 times.
Thread 3 attempting to withdraw 25 for 10 times.
Thread 4 attempting to withdraw 25 for 10 times.
Thread 2 attempting to withdraw 25 for 10 times.
Final balance:
Current balance: 0
```

Problem 3:

A Buffer class was created which consists of synchronised functions consume and produce for the private array of integers stored within it.

Additionally two threads were made Producer and Consumer which call the consume and produce functions in their run bodies, both access the same Buffer instance using reference.

Both the functions use

- **wait() -** to wait either until buffer is not full (for producer) or buffer is not empty (for consumer)
- **notify() -** to notify the producer/consumer that the operation has been completed.

In the screenshot to the right:
both the producer and consumer will be performing their operations 10 times. There is a sleep of 100 milliseconds between consecutive produce/consume.

If there was no synchronization, there would be many wasted produce items, where repeatedly a new item is trying to be added, but there is no space.

With synchronization we are preventing the waste of the operation by making sure it runs once it is possible, rather than letting unsynchronization let the operation go to waste.

```
student@student-VirtualBox:~/22CS02
Buffer empty. Consumer waiting...
Produced: 67
Consumed: 67
Buffer empty. Consumer waiting...
Produced: 69
Consumed: 69
Produced: 83
Consumed: 83
Buffer empty. Consumer waiting...
Produced: 85
Consumed: 85
Produced: 50
Consumed: 50
Buffer empty. Consumer waiting...
Produced: 67
Consumed: 67
Produced: 49
Consumed: 49
Produced: 20
Consumed: 20
Buffer empty. Consumer waiting...
Produced: 75
Consumed: 75
Produced: 41
Consumed: 41
student@student-VirtualBox:~/22CS02
```

Problem 4:

A Counter class was created which has a private integer variable, and a method called increment. The instance of this class is passed as a reference to the two threads which perform incrementation by 1 to the counter 100 times.

```
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q4
Final count: 200
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q4
Final count: 199
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q4
Final count: 200
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q4
Final count: 199
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$
```

As we can see without synchronization the final value can come out to be wrong, since it should always be 200.

We will add the synchronized keyword to the increment function of the Counter class to ensure only one thread can call it at a time, preventing race condition.

```
    public void increment() {
        count++;
    }
```

```
public synchronized void increment() {
    count++;
}
```

```
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q4
Final count: 200
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q4
Final count: 200
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q4
Final count: 200
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q4
Final count: 200
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q4
Final count: 200
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q4
Final count: 200
student@s Visual Studio Code ox:~/22CS02009/Object_Oriented_Programming_Laboratory$
```

This results in the Final count to come as 200 every time, removing the race condition.

Problem 5:

The SharedResource thread was created as per description, and a Worker class was created which calls the print() function of the SharedResource with its name.

Three threads were created with names HighPriority, NormPriority and LowPriority respectively and assigned the corresponding priorities which are predefined in the Thread Class.

```
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$ ./run Lab_9 Q5
HighPriority Thread is using the shared resource.
HighPriority Thread is using the shared resource.
HighPriority Thread is using the shared resource.
HighPriority Thread is using the shared resource.
HighPriority Thread is using the shared resource.
HighPriority Thread is using the shared resource.
HighPriority Thread is using the shared resource.
HighPriority Thread is using the shared resource.
HighPriority Thread is using the shared resource.
HighPriority Thread is using the shared resource.
LowPriority Thread is using the shared resource.
LowPriority Thread is using the shared resource.
NormPriority Thread is using the shared resource.
NormPriority Thread is using the shared resource.
LowPriority Thread is using the shared resource.
LowPriority Thread is using the shared resource.
LowPriority Thread is using the shared resource.
LowPriority Thread is using the shared resource.
LowPriority Thread is using the shared resource.
LowPriority Thread is using the shared resource.
LowPriority Thread is using the shared resource.
LowPriority Thread is using the shared resource.
NormPriority Thread is using the shared resource.
NormPriority Thread is using the shared resource.
NormPriority Thread is using the shared resource.
NormPriority Thread is using the shared resource.
NormPriority Thread is using the shared resource.
NormPriority Thread is using the shared resource.
NormPriority Thread is using the shared resource.
NormPriority Thread is using the shared resource.
student@student-VirtualBox:~/22CS02009/Object_Oriented_Programming_Laboratory$
```

We can see that the HighPriority thread is hogging the resource until it finishes, and LowPriority and NormPriority are somewhat fighting over the resources once the HighPriority thread is done.

Since only 10 iterations are being performed, we can extrapolate that in a system in which these threads will be running indefinitely, the HighPriority thread will dominate the access to the resource and Low and Norm Priority threads may starve,